HERENCIA MÚLTIPLE EN JAVA

Interfaces como mecanismo de herencia múltiple

Desarrollo de Sistemas Orientado a Objetos · 2026

Prof. Esteban Santini · 2026

¿QUÉ ES LA HERENCIA MÚLTIPLE?

Necesitar incorporar comportamientos de múltiples fuentes

En la programación orientada a objetos, la herencia múltiple ocurre cuando una clase necesita heredar atributos y métodos de más de una clase padre al mismo tiempo.

En lenguajes como C++, esta característica existe de forma directa. En Java, sin embargo, heredar de múltiples clases (concretas o abstractas) no está permitido.

Herencia múltiple: Una clase Pato necesita poder volar() como un Pájaro y nadar() como un Pez. Necesita heredar comportamiento de ambas fuentes. ¿Cómo lo resuelve Java?

La respuesta: mediante interfaces.

EL PROBLEMA DEL DIAMANTE

El escenario

Si las clases B y C heredan de A, y D hereda de B y C, surge una ambigüedad: si A define un método metodoA(), ¿qué versión hereda D: la de B o la de C?

Ejemplo:

```
   class A { void volar() {...} }  //
Pájaro
   class B extends A {...}          //
Águila
   class C extends A {...}          //
Paloma
   // ERROR: Java no permite esto:
   class D extends B, C {...}       // ¿?
```

La restricción en Java

Java tomó una decisión de diseño: una clase solo puede extender UNA clase padre directa. Esta restricción se denomina herencia simple.

¿Por qué?

Porque con múltiples clases padre que definen el mismo método, el compilador no puede determinar cuál implementación heredar: es ambiguo.

La solución: las interfaces solo declaran contratos, no tienen implementación propia que genere conflicto.

HERENCIA MÚLTIPLE EN JAVA

La solución: implementar múltiples interfaces

Java permite que una clase implemente múltiples interfaces al mismo tiempo. Esto otorga la flexibilidad del diseño de herencia múltiple sin los problemas de ambigüedad.

¿Por qué no hay ambigüedad con interfaces?

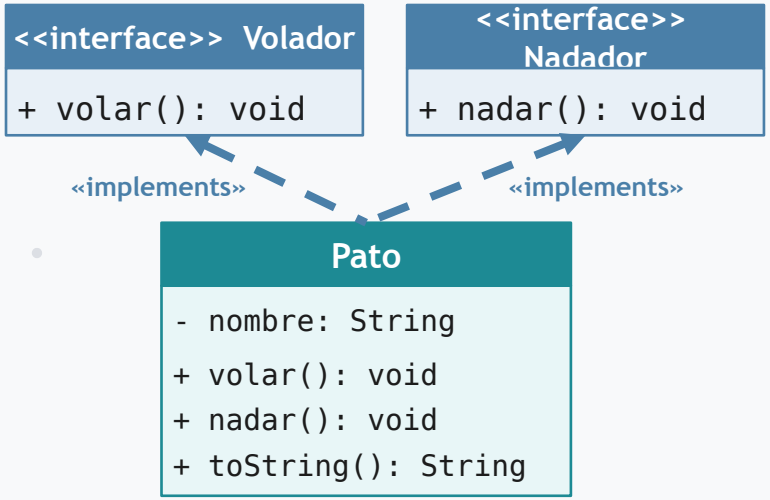
Las interfaces no tienen estado propio (no tienen atributos de instancia) y, salvo los métodos default, no contienen implementación. Solo declaran qué métodos debe proveer la clase.
La clase implementante es la responsable de escribir el cuerpo de cada método.

Sintaxis:

```
class Pato implements Volador, Nadador {
    // Pato DEBE implementar volar() y nadar()
}
```

NUESTRO EJEMPLO: La clase Pato

UML: Herencia múltiple via interfaces



Descripción del ejemplo

La clase Pato firma dos contratos simultáneos:

Volador: define el contrato volar()

Nadador: define el contrato nadar()

Pato es responsable de proveer una implementación concreta para cada método declarado en ambas interfaces.

Puntos clave:

Las interfaces NO tienen implementación propia.

Java puede resolver esta herencia sin ambigüedad.

Pato puede además extender otra clase (herencia simple).

CÓDIGO: Definiendo las interfaces



Ejemplo — interfaces Volador y Nadador

```
public interface Volador {
    // Todos los métodos son implícitamente public abstract
    void volar();
}

public interface Nadador {
    void nadar();
}

// Una interfaz define el QUÉ, no el CÓMO
// La clase que implemente decide el comportamiento concreto
```

¿QUÉ ES UNA INTERFAZ?

Un tipo de referencia que define un contrato

Una interfaz es una estructura especial en Java que define un conjunto de métodos que cualquier clase que la implemente DEBE proveer. No es una clase: no puede instanciarse directamente.

Características clave:

Todos los métodos son implícitamente public abstract (desde Java 8 pueden existir métodos default con implementación).

Los atributos son implícitamente public static final (constantes, no estado).

No tiene constructor ni estado de instancia propio.

Una clase puede implementar múltiples interfaces simultáneamente.

La interfaz define el QUÉ — la clase concreta define el CÓMO.

IMPLEMENTACIÓN DE UNA INTERFAZ

La palabra clave: implements

Para implementar una interfaz, la clase usa la palabra clave implements seguida del nombre.
La clase queda obligada a definir TODOS los métodos declarados en la interfaz.

Múltiples interfaces:

Se listan separadas por comas:
`implements Volador, Nadador`

Regla:

Si la clase no implementa TODOS los métodos declarados en la interfaz, debe declararse abstract.
El compilador lo verificará.

Ejemplo — 1 interfaz

```
interface Volador {
    void volar();
}

class Aguila implements Volador {
    @Override
    public void volar() {
        System.out.println(
            "Aguila vuela alto.");
    }
}

// El compilador verifica la implementación
```

CÓDIGO: Clase Pato



Clase Pato

```
public class Pato
        implements Volador, Nadador {
    private String nombre;
    public Pato(String nombre) {
        this.nombre = nombre;
    }
    @Override
    public void volar() {
        System.out.println(
            nombre + " vuela bajo.");
    }
    @Override
    public void nadar() {
        System.out.println(
            nombre + " nada en el lago.");
    }
}
```

Uso del objeto Pato

```
// Creamos el objeto
Pato p = new Pato("Donald");
// Invocamos por la interfaz Volador
p.volar();
// Donald vuela bajo.
// Invocamos por la interfaz Nadador
p.nadar();
// Donald nada en el lago.
// También: referencia por interfaz
Volador v = new Pato("Daisy");
v.volar();  // polimorfismo
```

CUADRO COMPARATIVO

Característica	Hereda clase concreta	Hereda clase abstracta	Implementa interfaz
¿Cuántas se pueden usar?	Solo una (extends)	Solo una (extends)	Múltiples (implements)
¿Hereda implementación?	Sí, completa	Sí, parcial (métodos abstract)	No (salvo default)
¿Obliga a implementar?	No	Sí, los métodos abstract	Sí, TODOS los métodos
¿Tiene atributos de instancia?	Sí	Sí	No (solo constantes)
¿Es instanciable directamente?	Sí	No	No

BENEFICIOS DE USAR INTERFACES

D Desacoplamiento

El cliente depende del contrato (interfaz), no de la implementación concreta. Cambiar la implementación no modifica el código cliente.

H Herencia Múltiple

Una clase puede incorporar comportamientos de múltiples fuentes sin el Problema del Diamante. Permite diseños altamente composibles.

T Testabilidad

Al programar contra interfaces, es sencillo sustituir implementaciones reales por mocks en pruebas unitarias.

E Extensibilidad

Agregar nuevas implementaciones no requiere modificar código existente. Compatible con el Principio Abierto/Cerrado.



LAS INTERFACES SON EL CONTRATO DEL DISEÑO

La clase implementa el acuerdo. La interfaz lo define.

Las interfaces en Java permiten la herencia múltiple de tipo sin comprometer la claridad. Desacoplan el diseño, habilitan el polimorfismo y promueven la extensibilidad del sistema.

Prof. Esteban Santini